# Spring Cloud Bus et Rabbit MQ

**Objectifs :**

1. Refresh à chaud avec Spring Cloud Bus
2. Spring Cloud Config Monitor

## Refresh à chaud avec Spring Cloud Bus

### Installer Rabbit MQ commer Docker

https://www.rabbitmq.com/docs/download

**docker run -it --rm --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:4.0-management**

- Ajoutez la dépendance suivante au pom.xml de tous les microservices :

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-bus-amqp</artifactId>
</dependency>
```

- Mettre à jour les fichiers application.yml de tous les microservices, en ajoutant sous la rubrique spring :

```
rabbitmq:
  host: "localhost"
  port: 5672
  username: "guest"
  password: "guest"
```

## Ajouter l'api author au projet Teacher

Copier la classe *Configuration*, à partir du projet *Departement*, dans le projet Teacher :

```
package com.nadhem.teacher.config;

import lombok.Data;
import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.stereotype.Component;

@Data
@Component
@ConfigurationProperties(prefix = "author")
public class Configuration {
   private String name;
   private String email;
}
```

Copier l'api  /author, à partir du projet *Departement*, dans le projet Teacher :

```
@Autowired
Configuration configuration;

@GetMapping("/author")
public ResponseEntity<String> retrieveAuthorInfo() {
   return ResponseEntity.status(HttpStatus.OK)
         .body(configuration.getName()+" "+configuration.getEmail() );
}
```

Tester l'api :

localhost:8081/api/teachers/author

## Tester le bus refresh

- Modifier et commitez sur github les fichiers department.yml et teacher.yml :

  author:

  name: "nadhem Teacher updated **Monday at 20:30**"

- Lancez un bus refresh sur un des micro services inscrits à rabbit MQ

  http://localhost:8080/actuator/busrefresh

- Testez : localhost:8081/api/teachers/author

On remarque qu'il suffit de lancer un busrefresh sur un seul  des microservices inscrits pour que tous les microservices seront mis à jour automatiquement en série.

# Spring Cloud Config Monitor

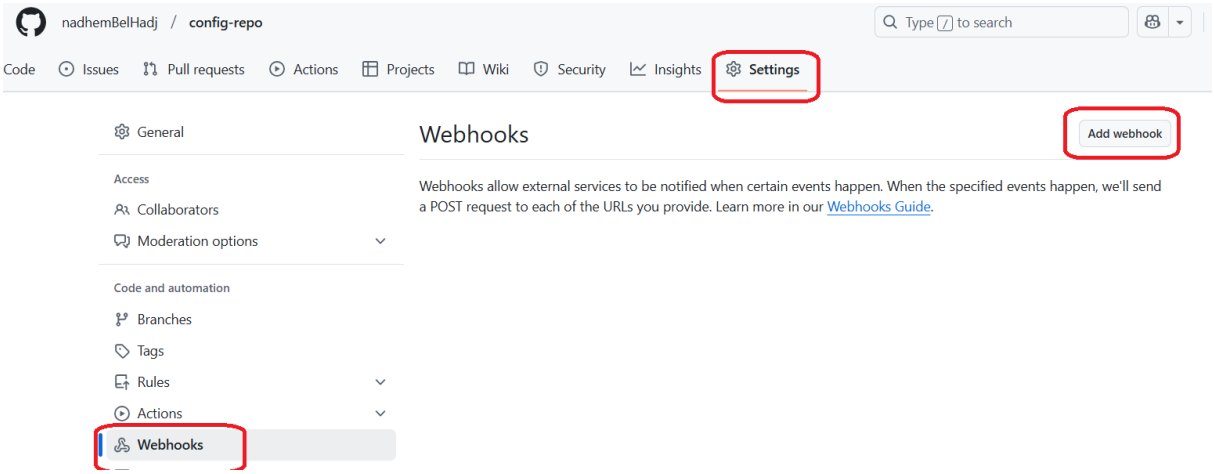
- Au niveau du projet ConfigServer, ajouter la dépendance :

```
<dependency>
   <groupId>org.springframework.cloud</groupId>
   <artifactId>spring-cloud-config-monitor</artifactId>
</dependency>
```

- Au niveau du projet ConfigServer, ajouter ces lignes au fichier application.yml, pour activer tous les end points de Actuator

```
management:
 endpoints:
  web:
   exposure:
    include: '*'
```

## Création d'un Webhook :



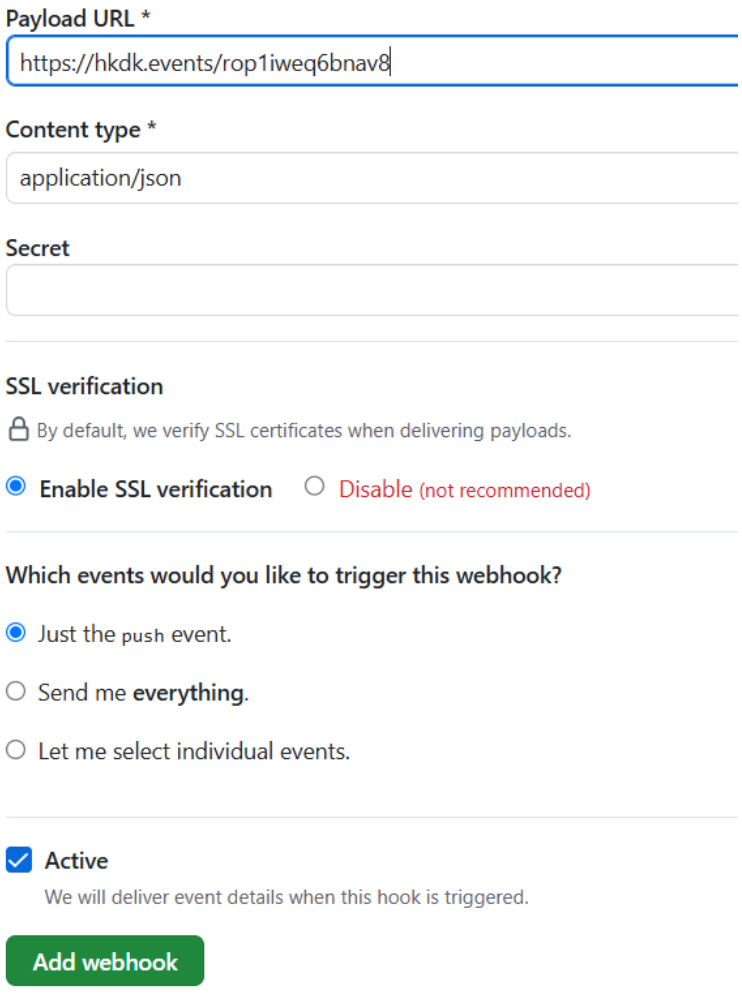
- Pour créer une adresse publique allez sur :

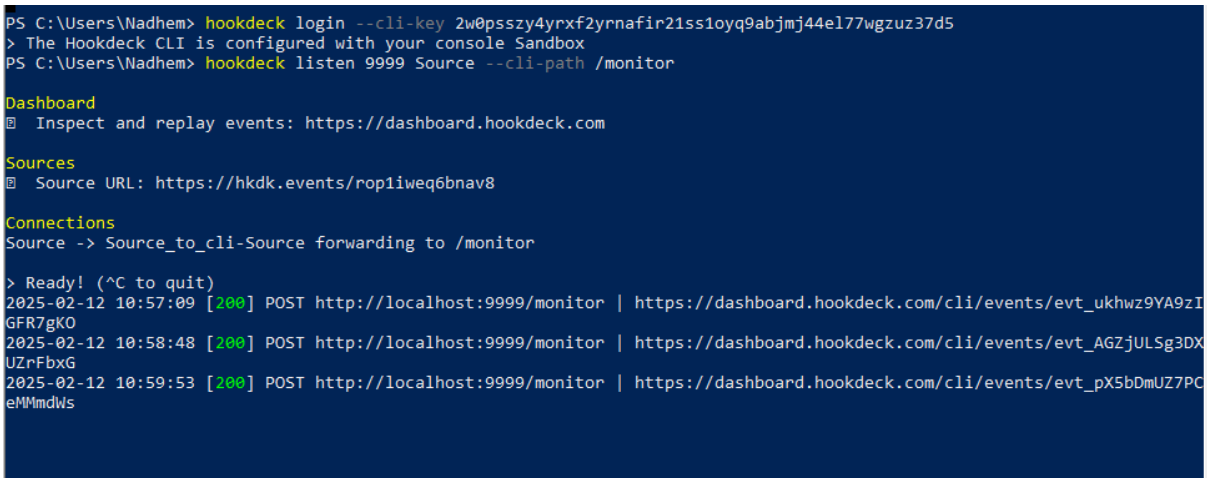
  https://hookdeck.com/

  Developpers/webhook console/Add Destination

- Installez scopp (Windows), Powershell puis lancez :
  - iwr -useb **get**.scoop.sh | iex
- scoop bucket add hookdeck https://github.com/hookdeck/scoop-hookdeck-cli.git
- scoop install hookdeck
- hookdeck login --cli-key 2w0psszy4yrxf2yrnafir21ss1oyq9abjmj44el77wgzuz37d5

- hookdeck listen 9999 Source --cli-path /monitor






Essayez le faire des modifications sur le fichier department.yml au niveau de GITHUB, commitez les puis lancez :

localhost:8080/api/departments/author

Vous allez remarquer que le refresh se fait automatiquement grâce à Webhook